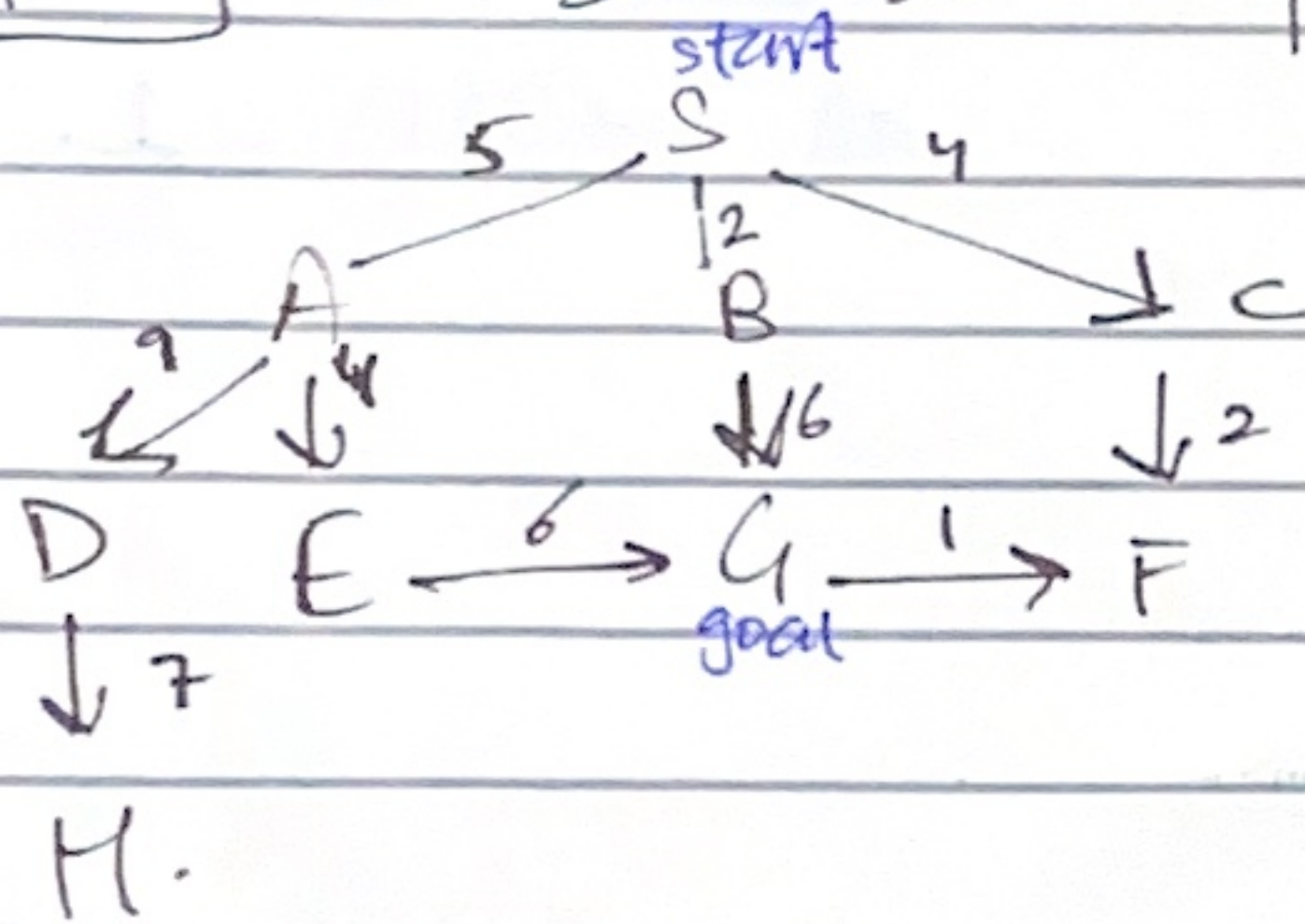


LECTURE-3.

Monday
26/1/25

BFS . Breather first search $\rightarrow$ Queue : level wise



Expanded Node

Frontier List (Queue)

	S	{S}
	A	{A _s , B _s , C _s } $\rightarrow$ from 'S'
	B	{B _s , C _s , D _A , E _A }
	C	C _s , D _A , E _A , G _B
popping	D	D _A , E _A , G _B , F _C
	E	E _A , G _B , F _C , H _D
	G (goal found)	↓ skip as 'G' is the only child of 'E' which is already in Queue G _B , F _C , H _D
		F _C , H _D

- Order of Traversal : $S \rightarrow A \rightarrow B \rightarrow C \rightarrow D$ (Completed)
- Elements in Queue when G (goal) found : F, H.
- Path (Backtrack): go from goal's subscript till start.
 $G_B \rightarrow B_s \rightarrow S$ $\therefore$ ~~$G \rightarrow B \rightarrow S$~~ $\downarrow$ is path.
- Path cost : add weights of path
 $2 + 6 = 8$ $S \rightarrow B \rightarrow G$

⇒ Activity in Sides: #26/80 AI Lecture 3"

• Convert table into graph.



Making Table:

Exp. Node	Frontier (Queue)
S	{S}
A _S	B _A , G _A
B _A	B_A, G_A G _A , C _A
G _A	G_A, C_A C _A , J _A
C _A	J _A , D _C , H _C
J _A	D _C , H _C , M _J
D _C	H _C , M _J , F _D
H _C	M _J , F _D
M _J	F _D , N _M
F _D	N _M , I _F
N _M	I _F , P _N
I _F	P _N , L _I
P _N	L _I , P_N
L _I	L_I, P_N P _N , K _L
P _N	P_N, K_L K _L , R _P
K _L	R _P , O _K
R _P	O _K , E _R
O _K	E _R → E_R

Branching factor: max. no. of children at any node

Depth: depth of shallowest node.

Maximum length: of any path

How many steps till goal

Time complexity

choices at each point

Path: E_R ← R_P ← O_K ← P_N ← N_M ← M_J ← J_A

S → A_S → G_A →

Path cost = 1 * 9 = 9

* If weights are 1 BFS gives least time

* For depth ≠ No. of elements in path 'q' in above case.

* Not good for large depthed algo. since in expo.

DFS: Stack → Ek corner park kr uski depth tk jayega.

* Graph on previous page S → A → ... H.

Exp. Node	Frontier Stack
S	{S}
A _S	A_S D _A , E _A , B _S , C _S
D _A	H _D , E _A , B _S , C _S
H _D	E _A , B _S , C _S
E _A	G _E , B _S , C _S
G _E ✓	B _S , C _S

path: G_E ← E_A ← H_D ← A_S ← S

5 + 4 + 6 = 15

* If we didn't make stack like $\frac{1}{n}$, then the path cost wld be low bcs path is: C → F → G ∴ path cost being optimal isn't always guaranteed.

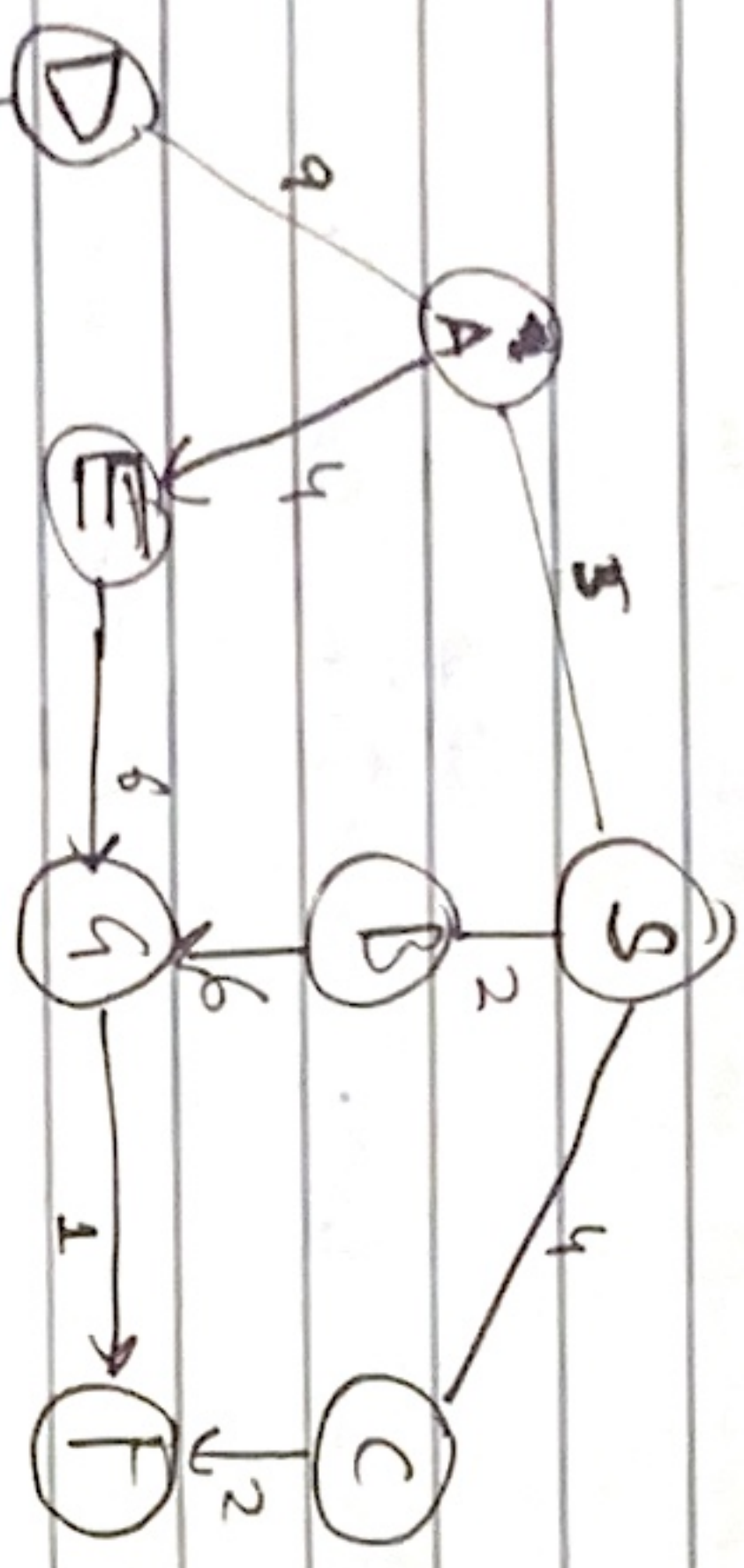
Initial state

Q	D	1	2
A		E	E
B		3	
C	5	4	

Numbers are dots to be eaten by Packman.
+ Alphabets are free paths.

LECTURE-4
Wednesday
28/11/26

UCS: Queue but we'll push the nodes in queue in increasing order of their weights like 2 3 5 9



* We'll not enqueue if the weight already in queue is less than a new path.
* weight from A → E isn't 4, it is $4+5=9$ as we're to add prev. path's weight as well.

Exp. Node	Frontier List (Queue)
S	B_5^2, C_5^4, A_5^5
B	C_8^4, A_5^5, G_8^8
C	A_5^5, F_6^6, G_8^8
A	$E_6^6, G_8^8, E_9^9, D_9^9$
F	$G_8^8, G_9^9, E_9^9, D_9^9$
G	G_8^8, E_9^9, D_9^9

Path = $S \rightarrow C \rightarrow F \rightarrow G$
cost = 7

① B was added first 8 then C as B had weight=2 and C has weight=4.
② enqueue F was child of C & should be added at end but since it's weight was 6 which is < 8 (weight of G), so in middle

path with fewer edges.

STORIER WAY: (for MCS)

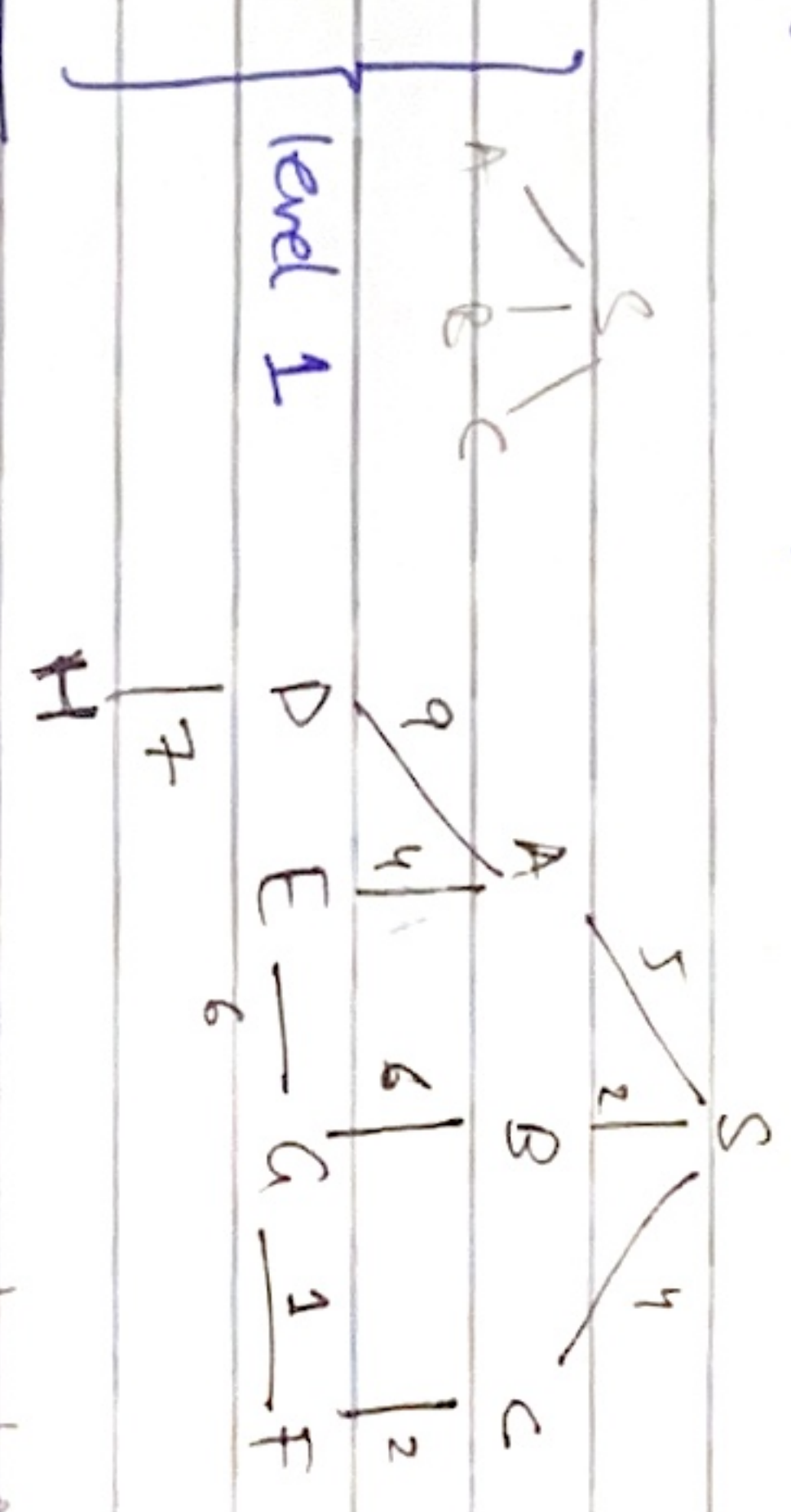
BFS: We'll find the route from $S \rightarrow G$ considering every weight = 1 & we'll find the path with least nodes in the path which is least cost.

DFS: We'll grab a node alphabet wise & then go to it's depth until we find goal in alpha. order.

UCS: We'll move to the nodes with least weight & keep going like that until goal giving optimal cost.
* Entire path should be optimal; not just neighbor.

IDS: Iterative Deepening search → Stack
We'll move level wise. First level 1 & see if goal found; keep increasing levels till goal found.

Exp. Nodes	Frontier
S	S
A	A_5, B_5, C_5
B	B_5, C_5
C	C_5
S	$\{ \}$
A	A_5, B_5, C_5
D	D_9, E_9, B_5, C_5
E	E_9, B_5, C_5
B	G_8, C_5
G	C_5



Path from level 2: $G \leftarrow B \leftarrow S$
Cost: $6+2=8$
+ only optimal
= 1 as S, C, F, G is better here -

Bi-Directional: searches from both ends.

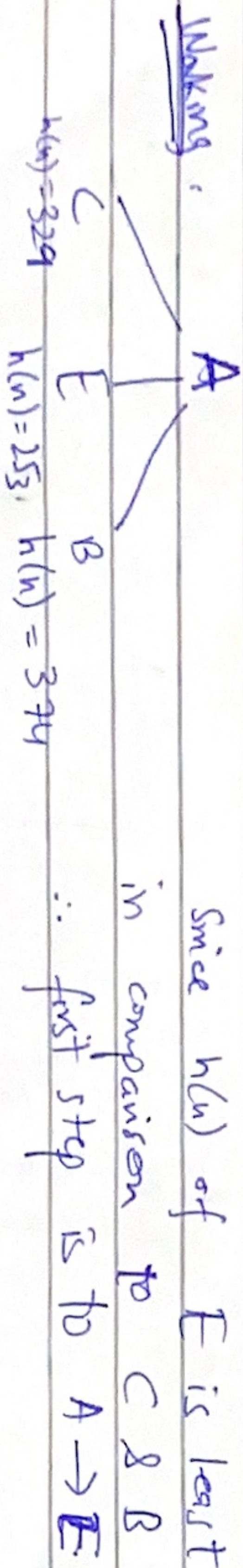
DLIS: Depth limited search. Better form of IDS.

If we're 100 levels, we'll have to search 100 levels in IDS but we can alter that here with user's input to break the process. Example we'll check first 25 level in one iteration, then 25-50 in second iteration then 75 & 100 in 3 & 4 iteration.

LECTURE - 5 Monday 21/2/28.

Informed Search: It has heuristic value (the value which tells displacement from goal to a point) $\Rightarrow$ Best-First Search.

Greedy BFS. Only considers $h(n)$ & moves towards graph & table in slide of lecture # 4. lower $h(n)$ value.



Continue... from slides. $A^* : f(n) = g(n) + h(n)$

$g(n)$ Cost = heuristic of final destination + weights till starting node.

Node	Frontier (Priority Queue)	
A	$E_{393}^A, B_{449}^A, C_{417}^A$	$B = 374 + 75 = 449$
E	$G_{413}^E, F_{417}^E, B_{449}^E, C_{457}^E$	$E = 253 + 140 = 393$
G	$H_{415}^G, F_{417}^G, B_{449}^G, C_{457}^G$	$G = 193 + 140 + 80 = 413$
H	$F_{417}^H, I_{418}^H, B_{449}^H, C_{457}^H$	$F = 178 + 140 + 99 = 417$
I	$(\text{ignore } I_{450})$	$\therefore G \rightarrow F$ (increasing order)

Path: $I \leftarrow H \leftarrow G \leftarrow E \leftarrow A$
Cost: 418 (better than greedy)

NOTE: For A^* to be optimal, heuristic func. should be admissible $\checkmark$

$\Rightarrow$ It is admissible if $h(n)$ value is equal to or less than weight (actual distance) on the final path which is optimal

ShortCut: If had to give path without solving; check if admissible then give shortestest path. Else give all available path.

Lecture - 6 Monday 4/2/28.

Finding heuristic value:

Q: we're two jans of 4 & 3 litres we need to find exactly 2 litres of water.

Initial state (4,3) // both jans filled
Final state (2,0) // one jan empty & 8 are has 2 liter.
Heuristic value $h(n) = \text{sum of direct distance from initial} \rightarrow \text{final}$
 $h(n) = |4-2| + |3-0| = 5$

weight = 1 (since not given)

operations:

- ① Completely full jar
- ② Completely empty jar
- ③ Pour from 1 to another.

heuristic value: Sum of abs distance.

two options right now only.

empty 1 or empty 2. (0,3) 5

we're not going to fill first as it will take us back in (4,3)

(4,3) 5

$|4-2| + |0-0| = 2$

(4,0) 2

(1,3) 4

(3,0) 4

(1,0) 4

(3,3) 4

(0,1) 3

(4,2) 4

(0,2) 4

(2,0) 0

Node Frontier (Priority Queue)

(4,3)

(4,0) 3, (0,3) 5

(0,0) 4, (0,3) 5, (1,3) 6

(0,3) 6

(3,0) 6

(1,3) 6

(1,0) 4

(3,3) 7

(0,1) 7

(4,2) 8

(0,2) 9

cost $f(n) = g(n) + h(n)$
weight which is 1
 $f(0) = 1 + 5 = 6$

Coordinates

H(n)

(4,3)

5

(0,3)

5

(4,0)

2

(1,3)

4

(3,0)

1

(3,3)

4

(1,0)

1

(0,1)

3

(4,2)

4

(4,1)

3

(0,2)

4

(2,0)

0

Adversarial Search.

Scenario: In Tic Tac Toe max (X) and min (O)

are playing.

Max has 9 options on first turn; min has 8 options;

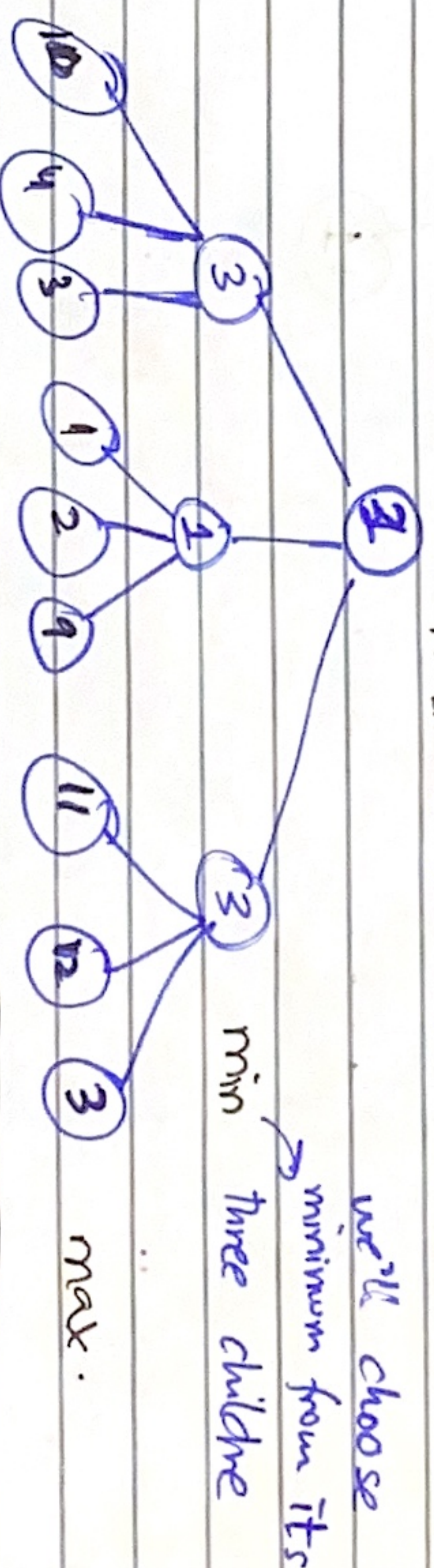
then max has 7 & so on...

Game end $\rightarrow$ Max wins (+1); Min wins (-1);

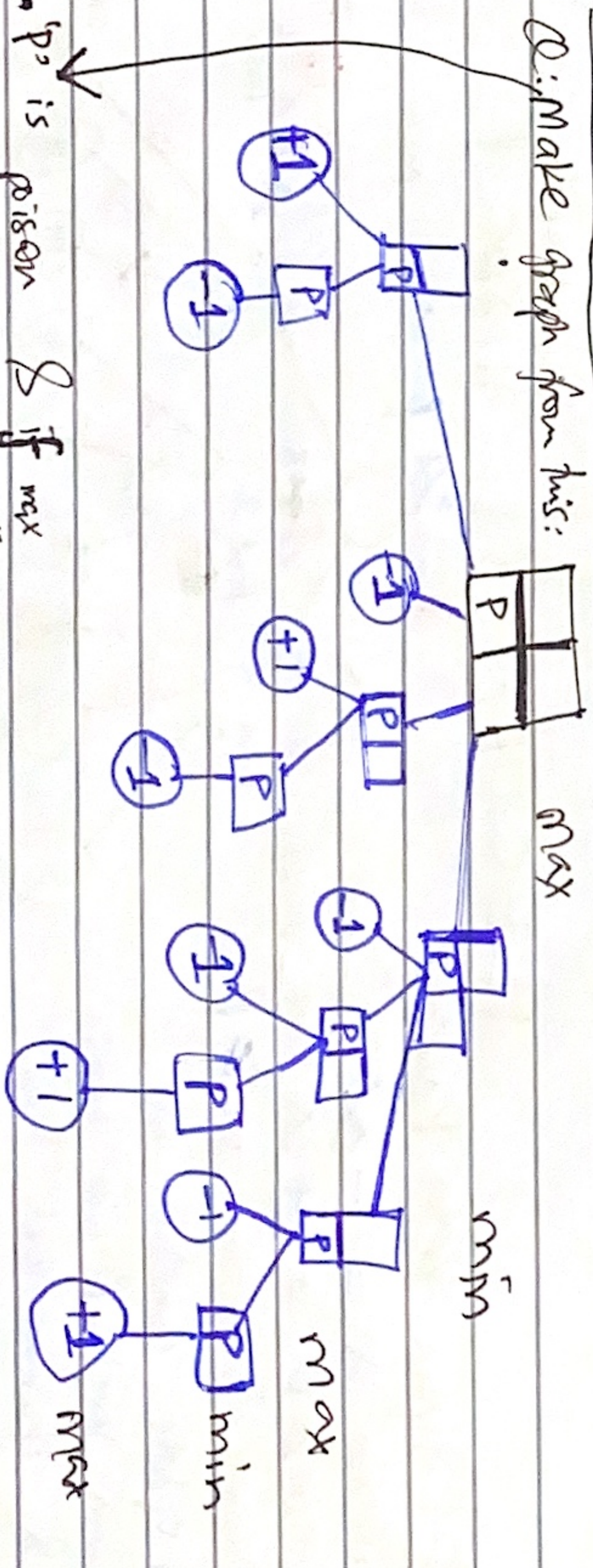
draw (0).

we'll choose maximum from better 3 child

we'll choose minimum from its three children



Q: Make graph from this:



'P' is poison

if max

at it, then min. will win hence

result = -1

Vice versa.

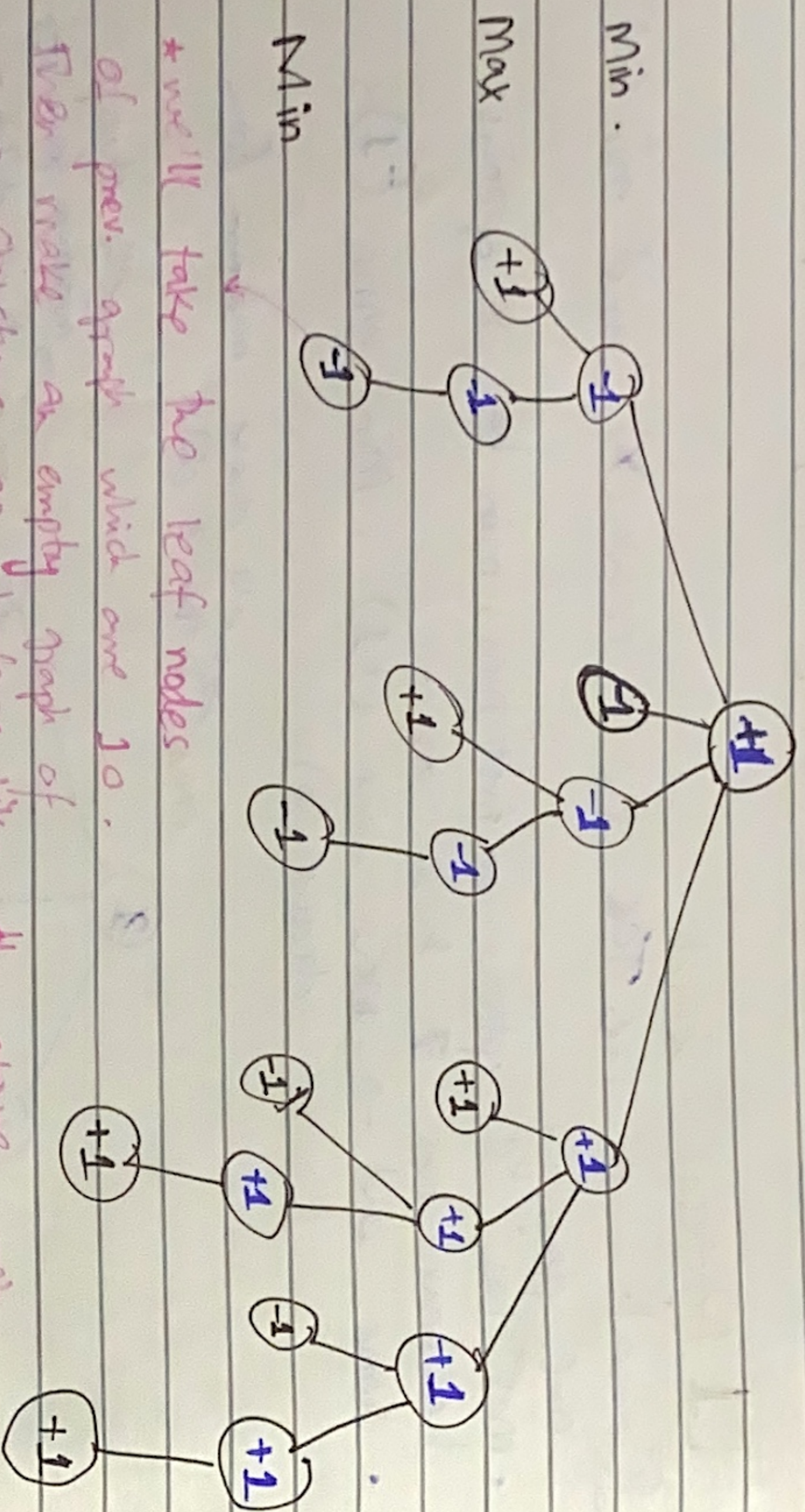
The chosen box along with it's above & right all blocks are gone.

LECTURE - 7

Monday

9/2/28.

Q Fill this based on prev. graph



* we'll take the leaf nodes of prev. graph which are 10. Then make an empty graph of same structure as before like the above graph. Then fill leaf with same numbers as prev. graph's leaf. Then just keep following min & max levels.

LECTURE - 8 Wednesday 11/2/26.

Alpha Beta Pruning.

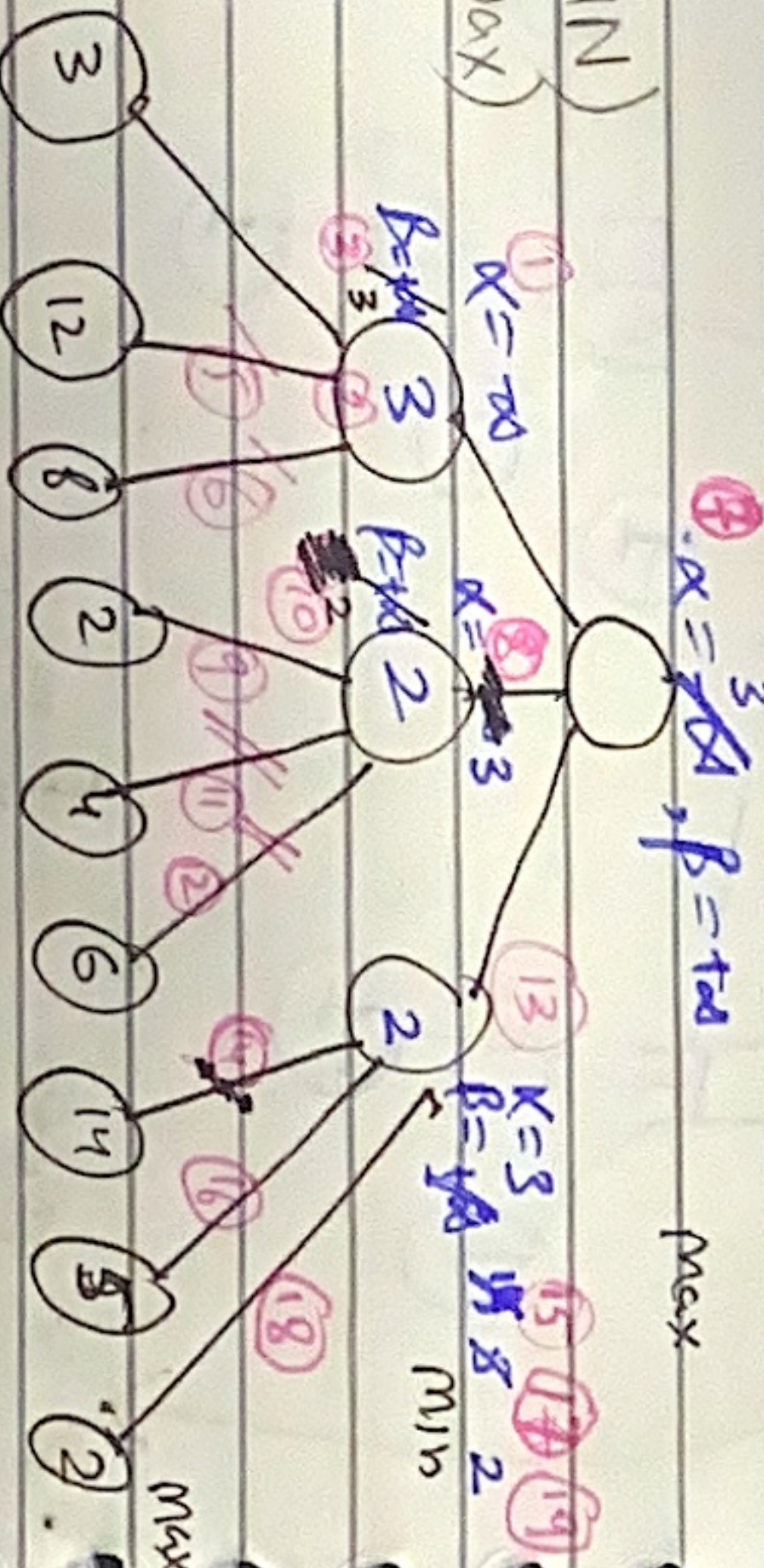
Max = $\alpha = -\infty$ (int - MIN)

Min = $\beta = +\infty$ (int + Max)

Pruning condition $\rightarrow \alpha \geq \beta$

* update something on min level; β will change

& if max level, then α change



Remove unnecessary paths from min max.

* At min level; we change β . At max level; we change α .

Always give α & β values to root. or grandchild. Then α, β will given to one of the child & we'll keep going deeper until the next level is of leaf nodes. While here we stopped at level 2 since level 3 is of leaf nodes.

First check if pruning condition (i.e: $\alpha \geq \beta$) meet or not. If not then

- 1 $\alpha = -\infty$, $\beta = +\infty$ goes to the child until the next node to that child is leaf node.
- 2 Move from left most leaf node i.e: 3 & 8 ^{copy} it above
- 3 Since $3 < +\infty$ so we'll change β with 3 by cutting too (as it's a min level so β changes).
- 4 check if the next leaf has lower value than '3'; since 12 & 3 $\therefore$ we'll ^{just update the node} ~~update the node~~ ^{as 3 is minimum} ~~update the node~~ ^{as 3 is minimum}
- 5 Similarly check next leaf; since 8 & 3 ~~update the node~~ ^{as 3 is minimum}
- 6 Since all 3 leaf nodes are cut; we'll move both α & β values to root (above level). Now as it's max level; so we'll change α with '3' & cut $-\infty$ since $3 > -\infty$.
- 7 Move $\alpha = 3$, $\beta = +\infty$ down the center branch.
- 8 Check first leaf & move its value to above node.
- 9 Since min. so modify β with '2' as $2 < +\infty$

Since it's a max level we're just concerned key kisi tarah nekey wala node sa higher value of α ~~are~~ ^{are} hoga. we'll ignore β .

Since pruning condition ~~met~~ ^{met} as ~~3 > 2~~ ^{$\alpha = 3, \beta = 2$} , 3 ≥ 2 $\therefore$ we'll prone (ELIMINATE) the rest two child branches of ②. This is the major alpha beta pruning. Double cut marked (11).

12 Branch eliminated as pruning condition met.

RN $\alpha=3, \beta=2$ on 2nd level; and $\alpha=3, \beta=+\infty$ on root.
 we'll move back to root and since it's max level so try updating α . Since alpha is already 3 which is greater than equal to '3 & 2' from below level; so no change.

13 Move $\alpha=3, \beta=+\infty$ to right must branch.

14 ~~Temporarily~~ Temporarily move to 2nd level and update $\beta=14$ (as its min level)

15 $\beta=14$

16 Temporarily move '5' to 2nd level as $5 < 14$.

17 $\beta=5$

18 Finally move 2 to the node as its the smallest.

19 $\beta=2 \dots$ Now $\alpha > \beta$ (as $\alpha=3, \beta=2$) so

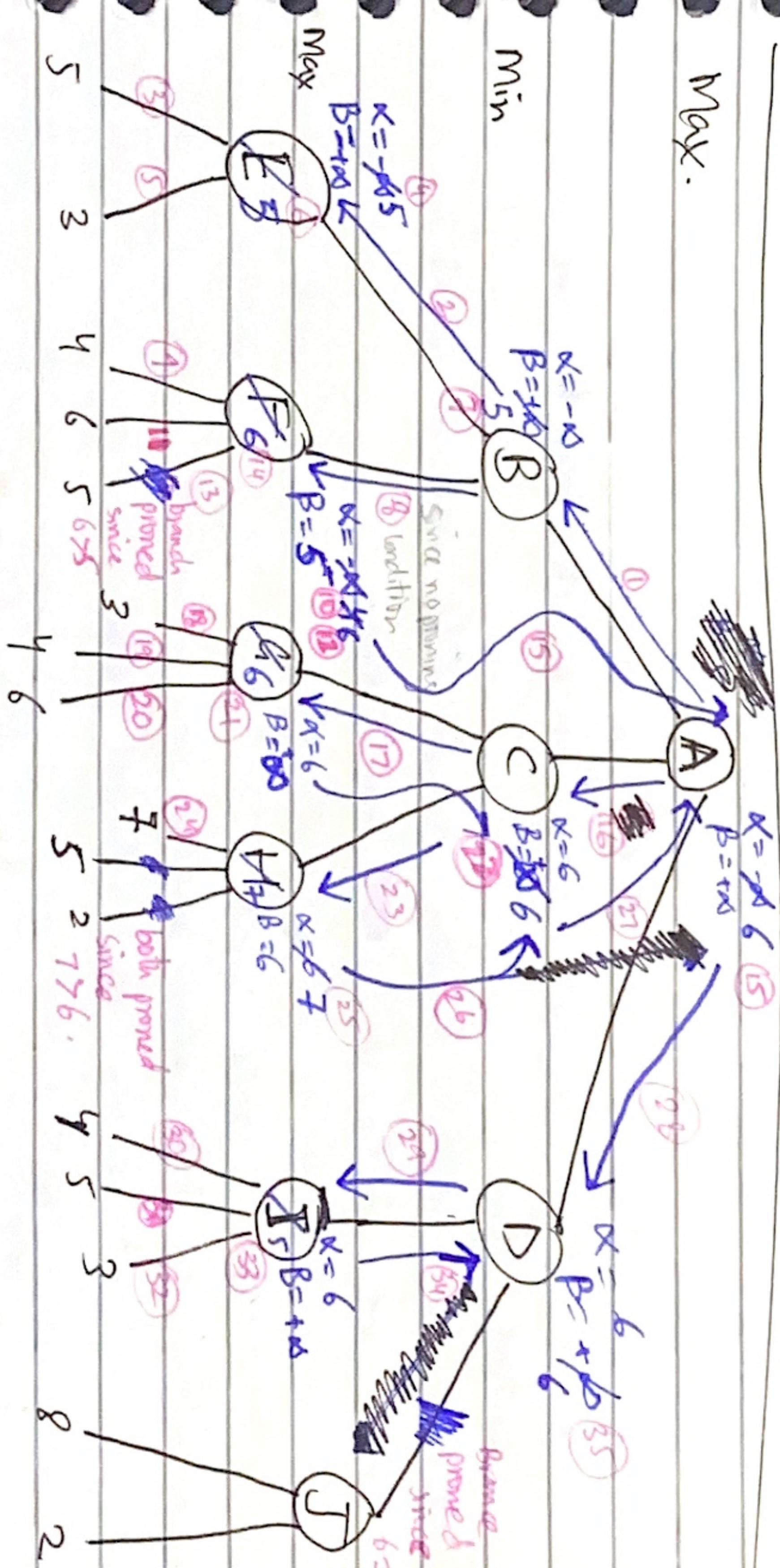
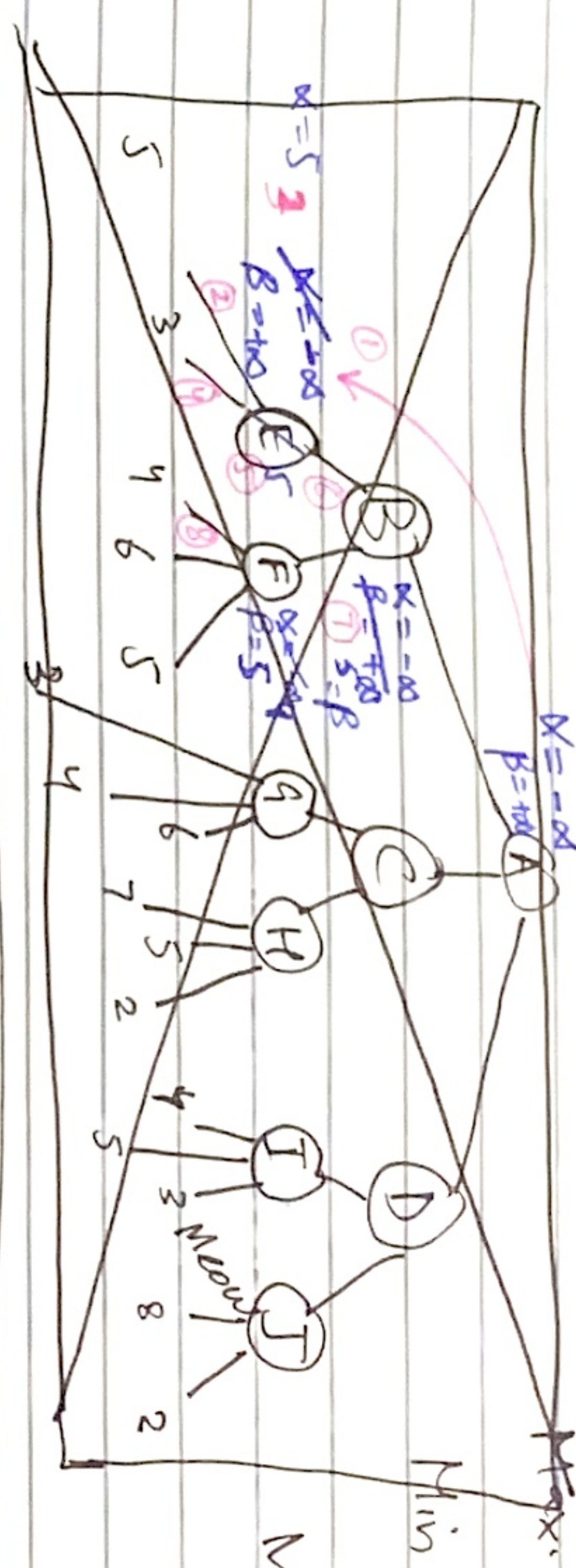
pruning condition met.

Since there are no other childs after it so no branches to eliminate.

NOTE: we won't set value from child to parent unless all nodes of that child aren't traversed.

Alpha Beta Prune Continue:

LECTURE-9 / Monday 16/2/20.



LECTURE - 10

Wednesday
17/2/25

Local Search: Reach a state where all

constants condition are fulfilled. Could be multiple final state. FOR OPTIMISE, not finding the exact path

- Path does not matter (e.g.: only end timetable with least clashes)
- Therefore; if only know the exact & its neighbouring nodes;
- No back track ~~its~~ possible as no track of path in start.

In overwords; it keep single current state and then it has a function which lets it move to any neighbour which brings it closer to goal state.

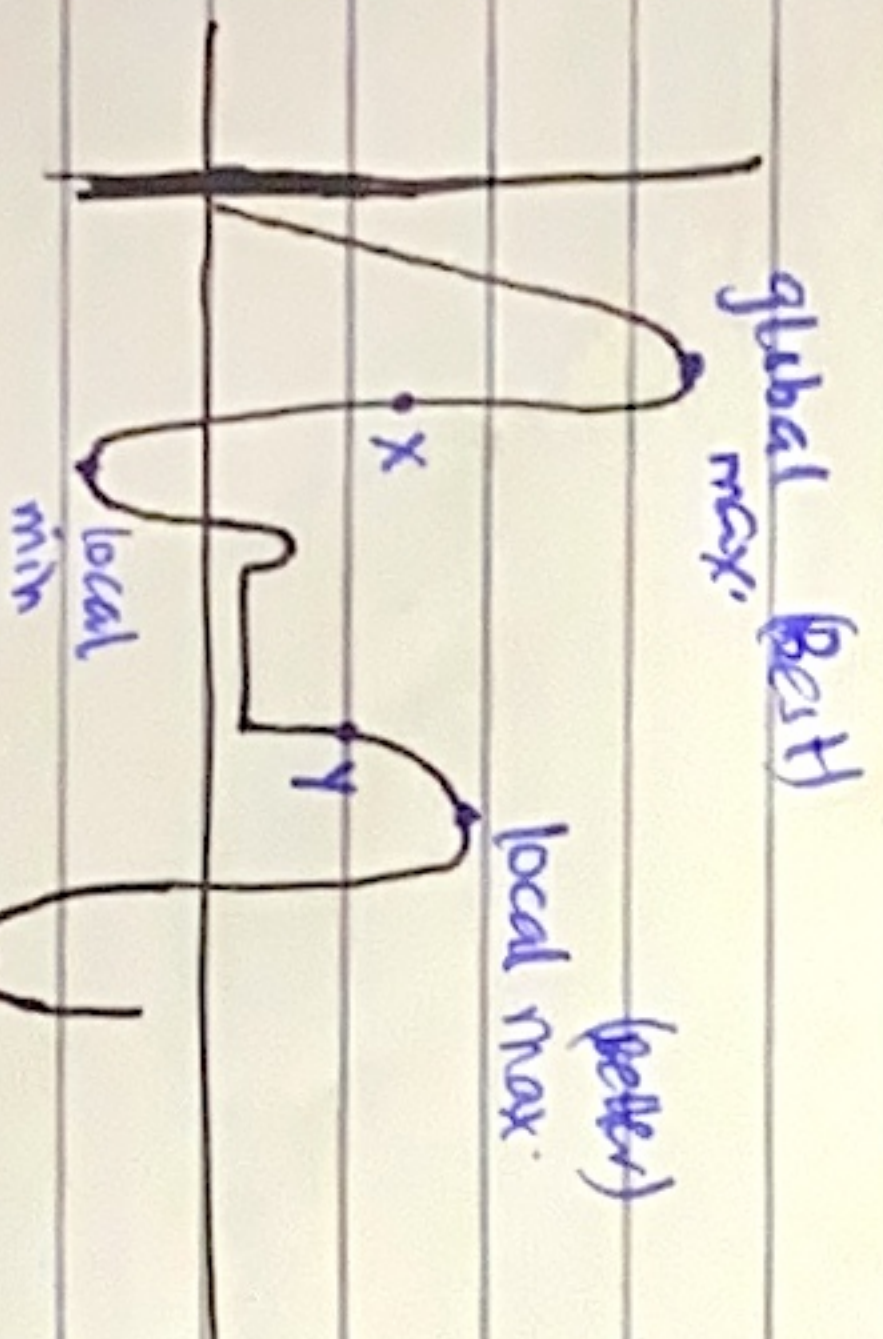
Steps:

- ① Generate a random starting state.
- ② Move to any neighbour which makes it closer to goal using evaluating func.
- ③ Repeat it until solution found OR no. of iteration which we defined ~~are~~ are done.

Advantage: No space used.

Disadvantage: • No back tracks

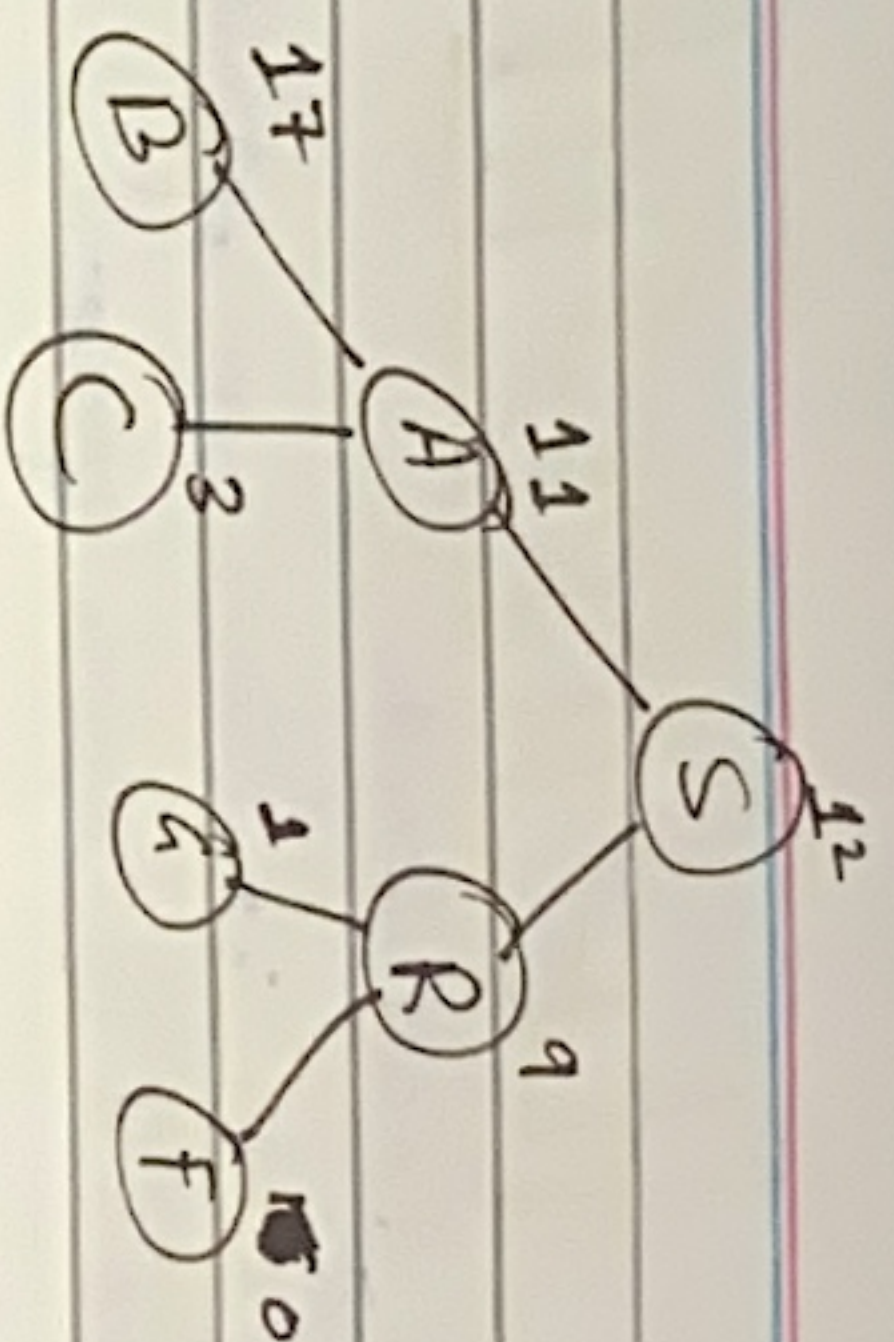
- Stuck in infinite loop



Making through graph:

A random starting point will be chosen like:

'y'. If the func. is for finding max; the algo will see which side of graph shld it move from y to go towards max. When it goes towards right from 'y'; it finds local max. and stop. If it started from 'x'; it wld go to global max.



⇒ Finding minimum: Suppose it started randomly from 'S'.

Now it will work like 'greedy BFS' by finding minimum values in its neighbours.

• It'll move towards R as $9 < 11$

• Then it will only remember its neighbours which are 'S' and 'F'?

• Now it will go 'F' from 'R' as $0 < 1$.

• Now it only remember its neighbours which are only R.

• But now it has forgotten that the path was from

$S \rightarrow R \rightarrow F$. That's the main diff. b/w 'greedy' & local search.

search.

• Now F is found which is by change 'Global minimum' (Best case).

⇒ Finding maximum.

• Starting from 'S'. It will try to find maximum value from neighbour.

• But now since the value of 'S' is greater than its neighbor as 12 is greater than A's & R's value (i.e. 11 & 9).

• Therefore it will now stay on 'S' making it local maximum.

NOTE: Therefore as we got local maximum & global minimum

So local search depends upon the random starting point.

LECTURE-11

Wednesday
4/3/25

Local Search.

Genetic Algorithms: focuses on optimization, not finding sol.

- For complex problems since a lot search space.
- The most fit/optimis path moves forward.

Population: set of possible sols.

⇒ Initial Population: Diversity in solutions for better output

⇒ Calculate fitness: Most stable output on any input is fitness func.

e.g: To convert 'Take' into 'make'; we've have a fitness value of '3' as 3 characters are matching & 3 are relevant to output.
From fitness func/value; we go to 'Selection'.

⇒ Selection: We'll choose 'Take' and 'also' instead of 'ABCD' as input since these two are having better fitness value & near to output.

⇒ Crossover & Mutation: Making of selected solutions from prev. step to move towards output. e.g: one way is to take first half of one solution & other half of the other. abcd & efgh → abcij & hijde

- We are creating new population to improve output

⇒ Mutation: Diversity & random changes

⇒ Convergence point: Point with max. fitness value.

Also called Chromosome.

e.g: '607' → 1001011111

[3,1,4,2] → shows which sites are visited

[1,0,3,4] → N Queens

Fitness Evaluation: For minimising func. value; we can choose value with lower answer from $f(n) = n^2 + 3n - 7$.

For Probabilities: Roulette wheel or Linear

Parent Selection Method: Find probability of all fitness values. Higher prob have better chance as a 'roulette' wheel

It's made like piechart with probabilities & the wheel spun with an arrow pointing; wherever wheel stops; that is the selected method.

Crossover: e.g on prev. page to combine two strings

Mutation:

LECTURE-12

Monday
4/3/25

Q: Maximise $f(n) = n^2$. where $n = [0, 31]$ (0 to 31)

Sr. No.	n	n-binary	f(n)	Parent Sldt	Expected Count	Actual Count
1	13	01101	169	$\frac{169}{1170} = 0.144$	$0.144 \times 4 = 0.576$	1
2	24	11000	576	0.49	1.96	2
3	8	01000	64	0.055	0.22	0
4	19	10011	361	0.308	1.232	1
			$\Sigma = 1170$			

5 genes
4 chromosomes
f(n) multiplied by 4 since initial population is 4.
Round up to whole no.

Actual Count: If we spin wheel four times; '11000' will occur two times, '01101' and '10011' will occur one time each and '01000' won't come at all.

Now we'll do crossover b/w Sr No. 1, 2 & 3, 4.

since 11000 occurred twice.

• Random numbers in array will be given like [0.1, 0.5, 0.3, 0.6]

Crossover (Single Point)

	n	$f(n) = n^2$
1 011011 → 010000	8	64
2 110000 → 111101	21	441
3 110000 → 110111	19	361
4 100111 → 100000	16	256
	$\Sigma n = 64$	$\Sigma f(n) = 1132$

★ Since $\Sigma f(n)$ is less than prev. (1132 < 1180) ...
Crossover wasn't helpful.

Choosing where to chop the binary for crossover..

• Take first value from random array i.e: 0.1

0.1 x 5 = 0.5 5 → chop after first bit for 182 pair

★ we've to see where to start from 0-4 or 1-5 bcs chopping on 5th is problem.

• For 2 & 4 pair; take next random value i.e: 0.5

0.5 x 5 = 2.5 3 → chop after 3rd bit.

Bit Flip

⇒ Mutation: Total genes = 5 x 4 = 20

• use next random no. from array i.e: 0.3

0.3 x 20 = 0.667 7 → 7th gene from a/b/c population

will be inverted. 0 → 1 or 1 → 0.

• Next random → 0.6 x 20 = 12th bit inverted.

01000
10101 → 10101 = 21
10011 = 19
00000

★ we did mutation
two times as mutation rate will be given
i.e: 0.1 in this case.
0.1 x 20 = 2

Linear Rank: Alt. of Roulette wheel. (for prob. fixing)

Sr.	n	$f(n)$	Best f(n)	E. Count	A. Count	Ranks
1	13	169	2/10 = 0.2			2
2	24	576	4/10 = 0.4			4
3	8	64	1/10 = 0.1			1
4	19	361	3/10 = 0.3			3
	$\Sigma = 1170$					$\Sigma = 10$

• this smoothes extreme values which were in roulette wheels. Have lowest prob. is '0.1' when there it was '0.05'.
• Gap reduces from hundreds to just single digit in rank, so fair -
higher no. = greater output

LECTURE-13 medias den (online) 11/3/26.

Elitism: Method for prob. selection.

• Transfer best individuals to next generation as it is w/out doing crossover or mutation. Every gen. will have some best indiv.

⇒ Multi-Point Crossover: • Suppose two points of intersection

11'0010'111

10'11111'00

Offs 1: 111111

Offs 2: 1001000

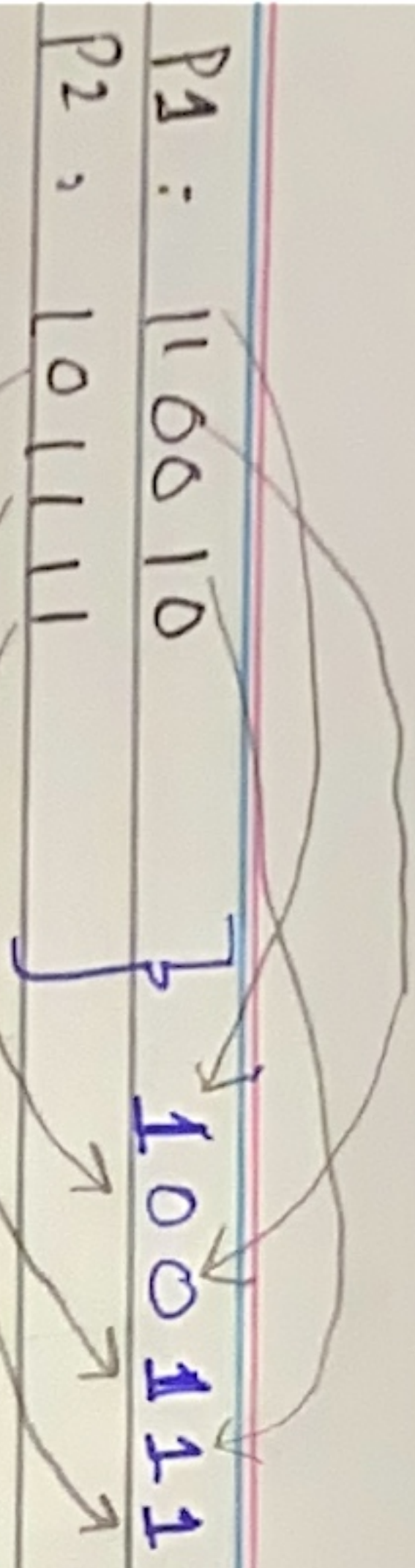
• swap the data b/w the two marked lines i.e: 0010 ↔ 1111
• rest on right & left data remain same

⇒ Uniform Crossover: For selecting whether to choose bit from parent 1 or 2 for offspring, coin is tossed each time.

• Has very high level of mixing since bits are very scattered.

• we'll pick the same index bit but is on mask to

choose b/w p1 & p2.



Mask: 101010 (rolling pin) 1: P1 bit selected
0: P2 bit selected

⇒ Arithmetic crossover: When we've digits in chromosomes instead of binary.

P1: [4, 8] for P1
P2: [10, 6] for P2
weight $\alpha = 0.5$, $\beta = 0.5$ (could be same weight for both)

Offs 1: $[0.5 \times 4 + 0.5 \times 10, 0.5 \times 8 + 0.5 \times 6]$
 $= [2 + 3, 4 + 1.8] = [5, 5.8]$
 It would be given in Ques. whether to round off, ceiling or floor.

⇒ Bit Flip Mutation: Done in prev. lecture $0 \rightleftharpoons 1$

⇒ Random Bit change: [103] generate a random no. & replace with any bit

⇒ Swap Mutation: Generate two random numbers & swap the bits on those two indexes of chromosome.

⇒ Selection Pressure: It guides the search towards optimal solutions by removing poor solutions & improves avg. fitness

- Higher selection pressure means faster evolution in species which reaches result faster BUT it can lead to premature convergence; means it can get stuck in local maxima/minima (like: Ultimatrix Aliens, Ben 10G)
- Low selection pressure means all sds. get same chances for crossover. Late finding the sol. but no premature convergence.

⇒ Generation gap: Difference of individuals that were in gen. A & are in B gen. after mutation; and also those were in A gen. & aren't in B gen.

Q: Population size = 10

- (i) Two offsprings replaced two old.
generation gap = $2/10 = 0.2$
- (ii) All 10 offsprings replaced prev. chromosomes
gap = $10/10 = 1$

Interpret: Ratio of change of population. ↑ rate ↓ iteration.

LECTURE - 14 Wednesday (online) 18/3/24

Machine Learning: Branch of AI that uses massive data to automatically learn patterns in it & improve from experience w/out being explicitly programmed.

⇒ Supervised learning: Labelled data

- Classification: Process of predicting a category/class. e.g. fruit name
- Regression: Predicting a ^{continuous} numeric value. e.g. Stock market price, marks.

⇒ Unsupervised learning: Unlabelled Data

- Clustering: Find patterns in unlabelled data based on similarities & differences. e.g. mango, oranges, apples could be classified in 3 groups on color, in 2 groups on shape or leaf/ventated.

LECTURE - 15

Wednesday (online)
25/3/28

Regression - Analysis: Studying & describing relationships b/w variables as it does more than prediction. It's responsibilities are:

- To understand which factors affect outcome
- How strongly variables are related
- Forecast future values
- Identify trends.

Types of Regression:

⇒ Linear Regression: Straight-line graph with one independent (on the basis of it, prediction is done) and one dependent (the thing we're predicting).
eg: House area (indep.) to House price (depend.)

Linear

⇒ Multiple Regression: One dependent & multiple independent variables.
eg: House price to House area, locality, no. of bedrooms etc.

⇒ Polynomial Regression: Non-linear regression for changing patterns.

eg: In young age, performance is stable & good & then it decreases.

⇒ Ridge & Lasso: Used when a lot of features. Have multiple lines in graph.

Q: Experience of Driver (x) Insurance Cost (y)

	x	y	x ²	xy
5	48	25	240	
2	47	9	94	
12	45	144	540	
9	40			
15	37			
6	35			
25	20			
16	25			
18	22			

$$\sum x = 108$$

$$\text{Mean}(x) = 12$$

$$\sum x^2 = 1720$$

$$\sum y = 319$$

$$\text{Mean}(y) = 34.44$$

$$\sum xy = 3295$$

* Regression coefficients are 'm' and 'c'.

$$\text{gradient } (m) = \frac{n(\sum xy) - (\sum x)(\sum y)}{n(\sum x^2) - (\sum x)^2}$$

$$m = \frac{9(3295) - (108)(319)}{9(1720) - (108)^2} = \frac{11115 - 34452}{15480 - 11664} = \frac{-23237}{3816} = -1.25708$$

$$y = mx + c$$

$$c = \bar{y} - m\bar{x}$$

$$c = 34.44 - (-1.25708)(12) = 50.524$$

eg. of Line of Best fit → $y = (-1.257)x + 50.524$

⇒ Regression Analysis of Graph:

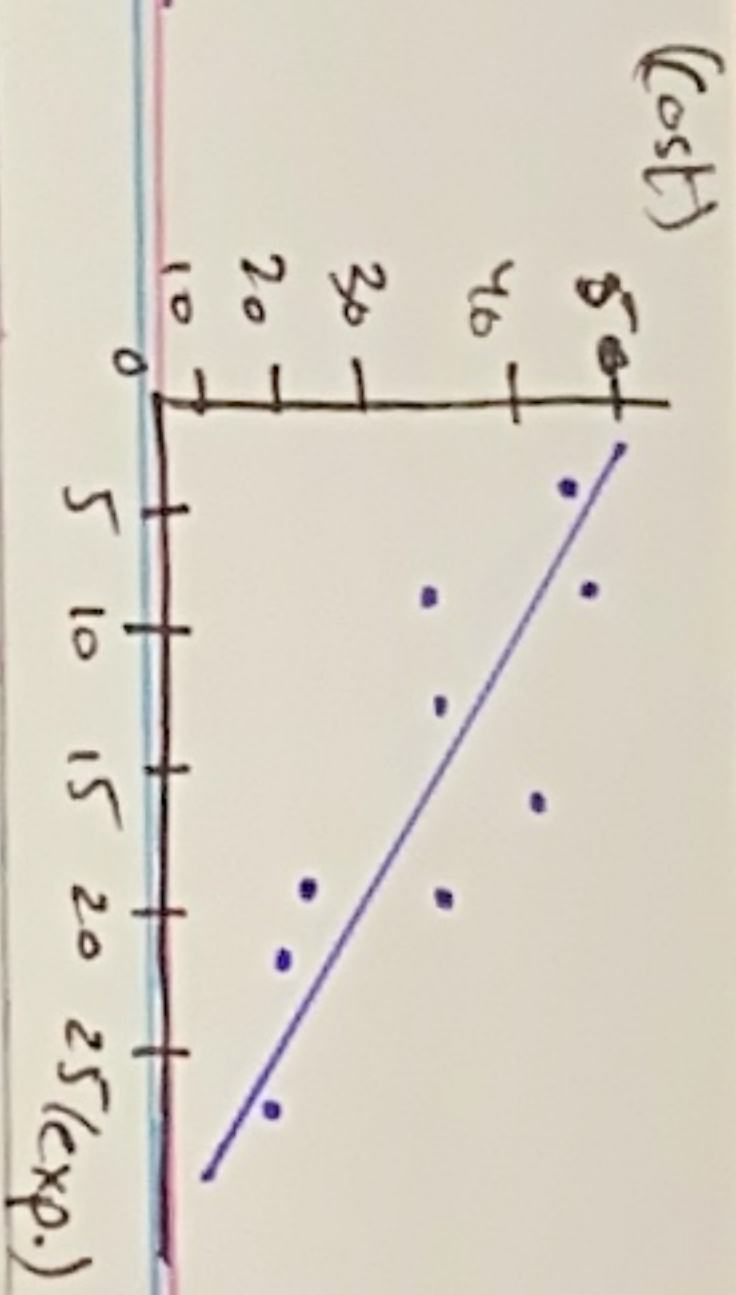
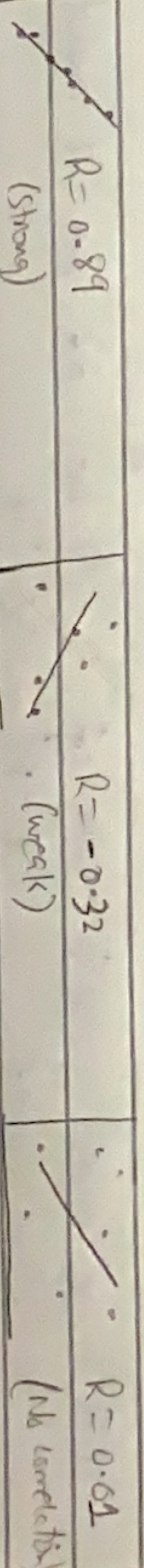
- Negative slope shows neg. correlation (if x ↓, y ↓ and vice versa).
- 1 unit increase in x (indep.) cause 1.25 decrease in y (depend.).
- Intercept = 50.524 (if no. experience of driver, then this is cost).

⇒ Performance Evaluation:

□ R²:

Coefficient of correlation (R): It tells how strong is the relation b/w dependent & independent variable.

- R being closer to +1 or -1 shows strong relation; close to zero shows weak relation (means the points in graph are far from Best Fit).



$$R = \frac{\sum (x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum (x_i - \bar{x})^2 \cdot \sum (y_i - \bar{y})^2}}$$

R of prev graph = 0.847 (negative strong correlation)

$$R^2 = (0.847)^2 = 0.717$$

- This is coefficient of determination. This shows that our model explains 71.7% of variation in output. In other words, how much of change in target variable (i.e. dependent variable (y)) is explained by input variables (i.e. independent (x)).
- 'Experience' explains 71.7% of insurance cost; rest 28.3% depends on other variables like ~~liscense~~ license, condition etc.
- ↑ R², good model. ↓ R², bad model

Mean Absolute Error (MAE): Tells how much error there could be in any value of 'y' predicted by model.

The error calculated is decided to be higher or lower on the basis of range of 'y' given. Eg: Mac=y is low error for range of 0-50 BUT it is higher for range: 10-20.

$$MAE = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|$$

↓
given value

→ Predicted value of 'y' using the eq. of best fit i.e. $y = mx + c$

$$MAE \text{ (of prev graph)} = 40.644/9 = 4.516$$

means if model predicts value of 'y' to be 32 then it could be in range 27.484 - 36.516.

Working of MAE for prev. graph.

x	y	y [^]	y - y [^]
5	48	44.339	3.761
2	47	48.01	1.01
12	45	35.44	9.56
9	40	:	:
15	37	:	:
6	35	:	:
25	20	:	:
16	25	:	:
18	22	27.898	5.898

$$\sum = 40.644 \rightarrow MAE = 40.644/9 = 4.516$$

$$y = (18)(-1.257) + 50.524$$

$$y = 27.898$$

Mean Square Error (MSE): Same as MAE but the absolute of $y - y^{\wedge}$ is replaced with square.

$$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

- Sensitive to outliers since bigger values are drastically increased bcs of ()².
- This helps when big values have serious effect on output as more weight given.

Root Mean Square (RMSE): Square root of MSE.

$$RMSE = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2} = \sqrt{MSE}$$

- Brings benefit of giving more weight to bigger values while bringing unit back to original (with $\sqrt{\quad}$)